

电子相框产品需求规格书

1. 硬件与显示规格

支持三种尺寸的六色墨水屏，数据存储于外部 Flash。

屏幕尺寸	分辨率	数据大小 (约)	存储位置	备注
3.7寸	480×720	168.75 KB	外部 Flash	六色显示
5.89寸	680×960	318.75 KB	外部 Flash	六色显示
7.3寸	无	无	无	分辨率未定，暂时只做占位

当前固件实际是 3.7 寸或 5.89 寸之一，7.3 寸仍是占位，没有固件实现。

2. 显示逻辑

状态	显示内容	显示逻辑说明
开机阶段	小程序二维码	若 Flash 内无图片，显示二维码；若 Flash 内有图片，跳过二维码显示图片。
关机时	LOGO 图片	显示保存在固件中的预设 LOGO。
运行阶段	照片内容	

3. 操作说明

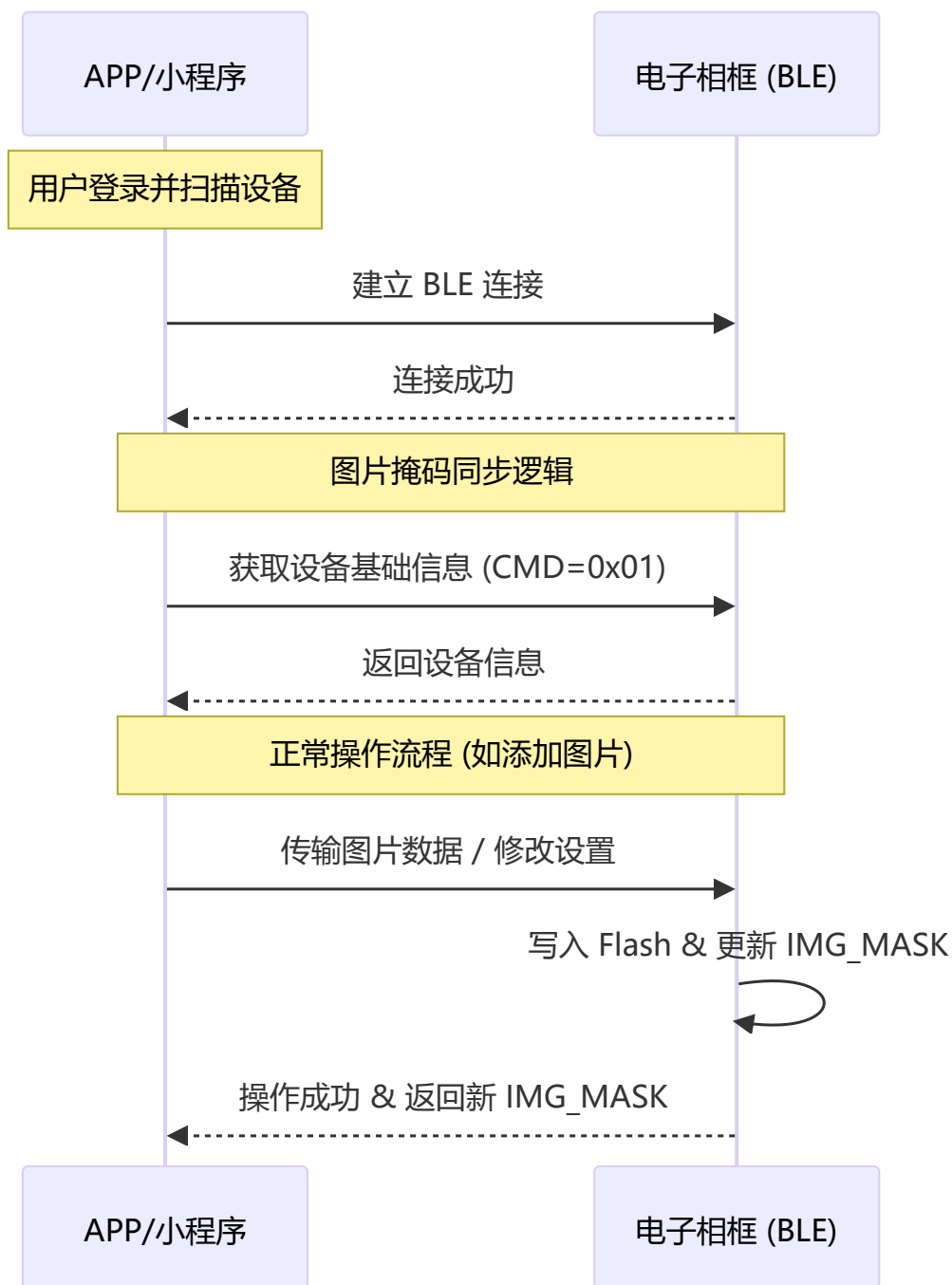
3.1 物理按键操作（单按键）

操作方式	功能定义	备注
长按 2s	开机 / 关机	
短按 1 次	切换照片	设备忙时不会响应
短按 3 次	删除当前显示的照片	设备忙时不会响应

4. 软件端（APP/小程序）交互逻辑

4.1 设备连接与数据同步

为了保证多端操作的数据一致性，系统采用**图片掩码**对比机制。



5. 固件与生产

5.1 生产与开发

- 二维码图片写入：固件硬编码存储
- 屏幕尺寸：由硬件型号决定

5.2 限制说明

- 图片掩码宽度：12 字节，96 位
- 最多图片数量：3.7 寸 95 张，5.89 寸 51 张
- 单张图片大小：根据屏幕尺寸，3.7 寸约 168.75 KB，5.89 寸约 318.75 KB

6. 通信协议与广播规范

6.1 系统架构概述

电子相框系统由两部分组成：

- 1. APP/小程序 - 用户界面，管理图片、与设备通信
- 2. 电子相框（设备） - 通过BLE与APP通信，管理Flash存储的图片，控制屏幕显示

通信流向： APP ↔ 设备（BLE）

6.2 通信方式与角色定义

6.2.1 通信方式总览

电子相框与 APP / 小程序之间通过 BLE 进行通信，采用"广播发现 + GATT 连接 + 应用层自定义协议"的方式实现设备发现、状态同步、图片传输、参数配置与固件升级。

6.2.2 BLE 角色定义

设备角色定义如下：

- 1. 电子相框
 - 作为 BLE Peripheral，负责广播、接收连接、处理命令、返回状态、接收图片数据并写入 Flash。
- 2. APP / 小程序
 - 作为 BLE Central，负责扫描设备、建立连接、读取设备状态、发起控制命令、传输图片。

6.2.3 设备连接与广播行为

设备采用单连接工作模式：

- 1. 设备仅开启一个可连接广播实例，用于被 APP / 小程序扫描发现。
- 2. 当手机端与设备建立 BLE 连接后，设备广播会自动关闭。
- 3. 设备处于已连接状态时不会继续广播，也不会接受第二个手机端连接。
- 4. 同一时间只能有一个手机端占用设备连接并进行业务操作。
- 5. 当前连接断开后，设备重新开启可连接广播，允许其他手机端重新扫描并连接。
- 6. APP / 小程序应将设备视为单连接资源：若扫描不到设备，可能表示设备已被其他手机端连接占用，需等待该连接断开后再重新扫描。

6.3 GATT 服务与特征定义

6.3.1 业务主服务

属性	值
Service UUID	0xFF00
用途	业务数据通信（配置、控制、状态上报）

特征定义

1. 命令下发特征（APP → 设备）

属性	值
Characteristic UUID	0xFF01
用途	APP 向设备发送命令包或数据包
属性	Write Without Response, Write

2. 状态/应答上报特征（设备 → APP）

属性	值
Characteristic UUID	0xFF02
用途	设备向 APP 返回命令响应、状态信息、业务数据
属性	Notify

6.3.2 OTA 服务（DFU）

属性	值
Service UUID	0xFF10
用途	固件升级（OTA / DFU）

特征定义

1. OTA 控制与数据特征（核心通道）

属性	值
Characteristic UUID	0xFF11
用途	OTA 控制指令与固件数据传输
属性	Write Without Response, Notify
说明	APP → 设备：发送 START / DATA 指令及固件数据 设备 → APP：返回 START ACK、DATA ACK 和最终校验结果

2. OTA 辅助通道 1（扩展/调试）

属性	值
Characteristic UUID	0xFF12
用途	扩展数据通道、调试信息
属性	Write Without Response, Notify

6.3.3 OTA 传输协议

OTA通道不使用 6.4 的 0xAA 应用层帧格式，直接在 0xFF11 特征上传输 OTA opcode 帧。所有 APP → 设备的 OTA 请求末尾都有 1 字节 CHECKSUM，计算方式为请求中除最后 checksum 外所有字节累加后取低 8 位。

DFU opcode:

名称	值	方向	说明
START	0xF1	APP → 设备	开始 OTA，携带对象类型和固件大小
DATA	0xF2	APP → 设备	首个 DATA 包为 128 字节 OTA 头信息握手包，后续 DATA 包为 bin 数据分片
END	0xF3	APP → 设备	APP 在 bin 数据发送完成后发送结束包，设备校验整包并返回 ACK
ACK	0xFC	设备 → APP	START / DATA / END 的应答帧头

DFU 结果码:

值	含义
0x00	成功
0x01	校验 / 芯片信息错误
0x02	ACK 超时
0x03	主机主动终止
0x04	设备主动终止
0x05	设备状态错误
0x06	不支持的 opcode
0x07	资源不足
0x08	固件大小超限
0x09	参数错误
0x0A	协议匹配字段不匹配
0x0B	头部版本不兼容
0x0C	固件类型与面板不匹配
0x0D	头部 CRC16 校验失败
0x0E	版本号校验失败

START 请求:

字段	字节数	说明
OPCODE	1	固定 0xF1
OBJ_TYPE	1	对象类型，当前作为固件对象使用
FW_SIZE	4	固件大小，小端；范围 0x3000 ~ 0x3C000
CHECKSUM	1	前 6 字节累加和低 8 位

START 响应：

字段	字节数	说明
RSP_CODE	1	固定 0xFC
OPCODE	1	对应 0xF1
RESULT	1	OTA 结果码
MTU	1	固件返回 251
CREDIT / PRN	1	固件返回 3，APP 每发送 3 个 DATA 包等待一次 ACK
CHECKSUM	1	前 5 字节累加和低 8 位

6.3.4 OTA 第一包协议握手

1. 概述

OTA 升级分为两个阶段：

阶段	说明
握手	第一个 0xF2 包，128 字节头信息，设备校验后回复结果
传输	后续 0xF2 包为 bin 数据，最后 0xF3 结束，校验整包 CRC32

2. 握手请求：头信息包（128 字节）

说明：协议沿用现有 DFU 指令，opcode = 0xF2 (DATA)，设备端通过首包长度判断为头信息。

该 128 字节头信息已经位于升级包文件最前端，升级包整体结构为 [128 字节头信息][原始 bin payload]。APP / 小程序只负责按文件内容读取并发送。

偏移	长度	字段	说明
0	8	协议匹配字段	固定 "OTAINFO\0"，ASCII
8	1	头部格式版本	当前 1
9	1	固件类型	1 = 3D7 面板，2 = 5D89 面板
10	2	头部长度	固定 128（0x80 0x00），小端序
12	4	原始 bin 文件大小	不含本 128 字节头，小端序

偏移	长度	字段	说明
16	4	原始 bin CRC32	不含本头的 payload CRC32，小端序
20	2	版本号字符串长度	<= 104，小端序
22	2	头部 CRC16	覆盖偏移 0~21 + 24~127，小端序
24	len	版本号 ASCII 字符串	格式见 §三
24+len	104-len	填充 0x00	

3. 版本号格式

按  拆分为 6 段：

段	含义	规则	示例值
①	产品前缀	固定 BR1601A02	BR1601A02
②	构建日期	6 位数字，月 01-12，日 01-31	260702
③	SVN 版本	r + 4 位数字	r8285
④	项目号	固定 5139	5139
⑤	面板型号	3D7 或 5D89	5D89
⑥	固件版本	v + 3 位数字	v100

完整示例：

```
BR1601A02_260702_r8285_5139_5D89_v100
```

偏移 24 起写入（十六进制）：

```
42 52 31 36 30 31 41 30 32 5F 32 36 30 37 30 32  BR1601A02_260702
5F 72 38 32 38 35 5F 35 31 33 39 5F 35 44 38 39  _r8285_5139_5D89
5F 56 31 30 30 00 00 00 00 00 00 00 ... 00      _v100(填充)
```

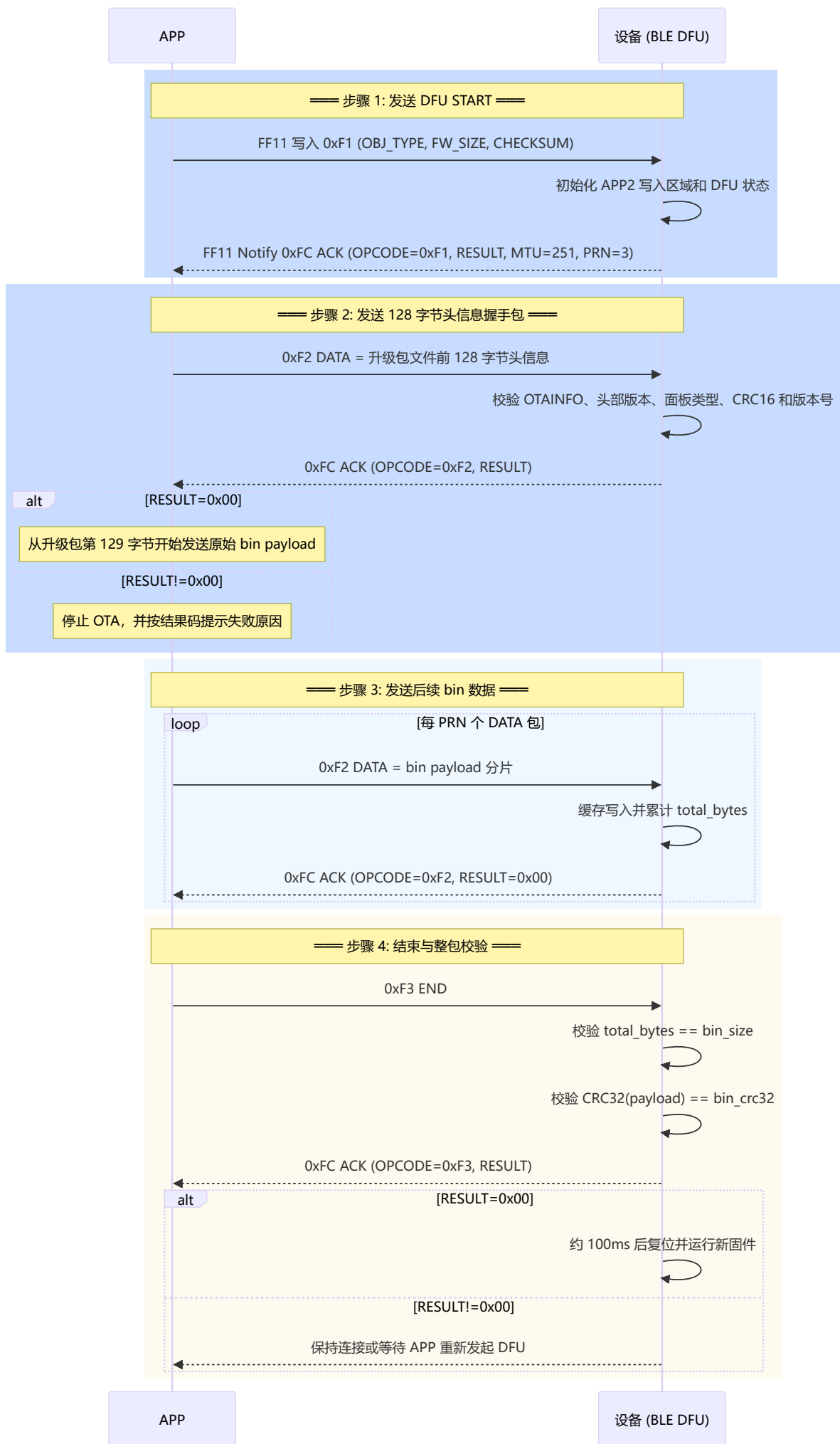
6. 结果码

值	错误码	含义	常见原因
0x00	SUCCESS	握手成功	准备接收数据
0x0A	BAD_PROTOCOL	协议匹配字段不匹配	非 OTA 包或文件损坏
0x0B	HEADER_VERSION	头部版本不兼容	工具链需升级
0x0C	WRONG_PANEL	固件类型与面板不匹配	5D89 固件→3D7 设备
0x0D	HEADER_CORRUPTED	头部 CRC16 校验失败	传输丢包或文件截断
0x0E	VERSION_INVALID	版本号校验失败	格式/产品/面板不一致

7. 数据接收阶段（与现有协议相同）

opcode	含义	设备动作
0xF2	数据包	后续 F2 包不再校验长度，直接缓存写入
0xF3	结束包	校验 <code>total_bytes == bin_size</code> ，校验 <code>CRC32(payload) == bin_crc32</code> ，均通过回 <code>0x00</code> ，否则回 <code>0x09</code>

8. DFU 传输流程图



6.4 应用层协议帧格式

6.4.1 基本帧结构

字段	字节数	说明
SOF	1	帧头，固定 0xAA
CMD	1	命令字，区分业务命令
LEN	2	负载长度（小端），PAYLOAD 的字节长度
PAYLOAD	N	可变长度，具体业务数据
CRC16	2	CRC16-Modbus（小端），对 SOF ~ PAYLOAD 校验

协议传输参数:

参数	值	说明
MTU	247	BLE协议层请求的最大传输单元。实际GATT可用244字节。
Credit	5	一次可发送最多5个数据包，之后必须接收一次应答
超时时间	1s	APP 如果超过 1 秒未接收到应答包，或设备超过 1 秒未收到下一包数据，均视为传输中断

6.5 命令字定义

6.5.1 信息获取类（0x01、0x02、0x03、0x04、0x05）

命令字	功能说明
0x01	获取设备基础信息
0x02	获取播放配置
0x03	获取软件版本
0x04	获取电量信息
0x05	获取连接间隔

6.5.2 配置设置类（0x10、0x11、0x13）

命令字	功能说明
0x10	设置播放配置
0x11	设置系统时间
0x13	设置连接间隔

6.5.3 图片控制类 (0x12、0x24)

命令字	功能说明
0x12	删除图片
0x24	屏幕刷新 (指定显示的图片索引)

6.5.4 图片传输类 (0x20~0x23)

命令字	功能说明
0x20	开始传输 (图片帧头)
0x21	传输图片数据包 (分片装载)
0x22	结束传输 (文件级CRC校验)
0x23	图片传输应答 (累计确认)

6.5.5 系统类

命令字	功能说明
0x7F	通用应答

6.6 应答格式

根据命令类型，系统采用两种应答格式：

6.6.1 通用应答 (CMD = 0x7F)

用于 0x01、0x02、0x03、0x04、0x05、0x10、0x11、0x12、0x13、0x20、0x24 等非图片数据流命令。

PAYLOAD：

字段	字节数	说明
ACK_CMD	1	对应原命令字
RESULT	1	结果码 (见下表)

RESULT 结果码定义：

值	含义
0x00	成功
0x01	参数错误
0x02	长度错误
0x03	CRC 错误

值	含义
0x04	Flash 写入失败
0x05	图片不存在
0x06	存储空间不足
0x07	掩码不一致
0x08	不支持的命令
0x09	传输中断
0x0A	校验失败
0x0B	设备忙 (Busy)

当前固件包含 `epf_service` 忙状态保护：忙状态下只允许读取 `0x02` 播放配置、`0x03` 软件版本和 `0x05` 连接间隔；图片传输开始后只接受 `0x21` 数据包和 `0x22` 结束包；其他写类命令在设备忙时返回 `0x0B`。

6.6.2 图片传输应答 (CMD = 0x23)

仅用于 `0x21` 传输图片数据包的确认，采用**累计确认**机制。

PAYLOAD:

字段	字节数	说明
ACK_SEQ	2	已连续接收到的最后一个数据包的序号

处理逻辑:

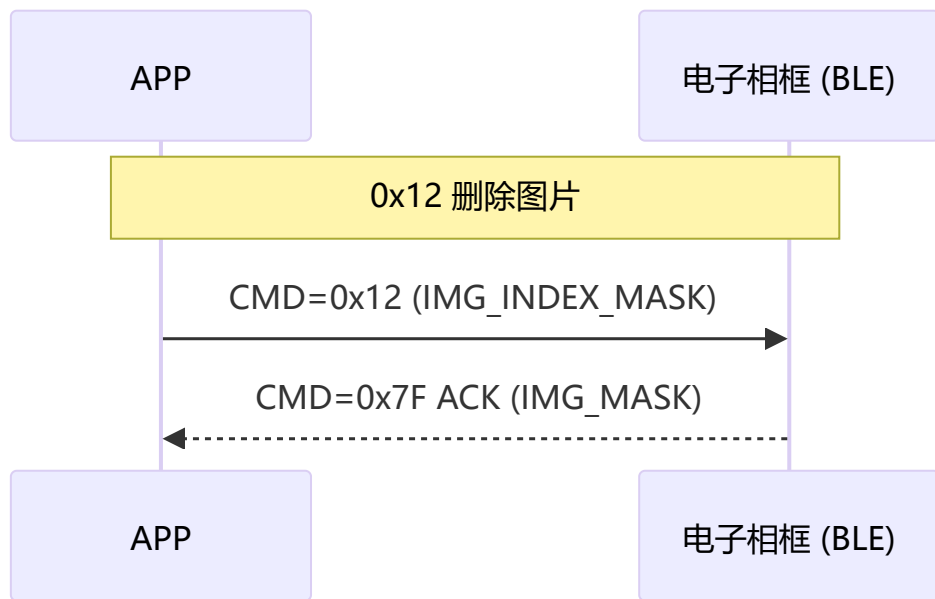
设备仅需返回已经连续接收到的最后一个数据包序号。APP 根据 ACK_SEQ:

- 计算下一个要发送的包号: `next_seq = ACK_SEQ + 1`
- **包丢失检测:** 若 ACK_SEQ 无进展 (保持不变), 说明期望包有丢失, APP 检测到后立即重新发送从 next_seq 开始的包

6.7 关键命令协议定义

6.7.1 通用查询/设置命令流程





6.7.2 图片管理策略

系统采用 **12 字节图片掩码** 直接标识和管理图片。每个图片占用 `IMG_MASK` 中的一个比特位（位 0~位 95），无需额外的 ID 结构。

图片管理原则：

1. **位索引标识** - 用 0-95 的位索引直接标识对应图片，APP 和设备通过相同的位位置来识别
2. **掩码操作** - `IMG_MASK` 中的每一位代表对应位索引的图片存在状态（1=存在，0=不存在）
3. **简洁高效** - 无需额外的结构化数据，直接通过位运算即可完成图片管理
4. **一致性同步** - APP和设备通过对比`IMG_MASK`中的比特位来确保图片一致

6.7.3 获取设备基础信息（CMD = 0x01）

请求包 PAYLOAD：空

响应包 PAYLOAD：

字段	字节数	说明
Device_ID	6	设备唯一编号，建议使用 BLE 地址或 SN 映射值
HW_VER	2	硬件版本
IMG_MASK	12	图片掩码（96位，每位对应一张图片，1=存在，0=不存在）
BATTERY	1	电量百分比，范围 0 ~ 100
PLAY_MODE	1	播放模式：0x00 顺序，0x01 随机，0x02 手动
INTERVAL	4	切换间隔，单位秒
SCREEN_TYPE	1	屏幕类型：0x01 3.7寸，0x02 5.89寸，0x03 7.3寸
CUR_IMG_INDEX	1	当前显示图片索引，范围由屏幕容量决定；无图片时为 0xFF

示例：

请求: AA 01 00 00 [CRC16]

响应: AA 7F 1E 00

```
12 34 56 78 9A BC      // Device_ID (6字节)
01 00                  // HW_VER (2字节) = 1
03 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 // IMG_MASK (12字节)
52                    // BATTERY = 82%
00                    // PLAY_MODE = 0x00 (顺序播放)
3C 00 00 00           // INTERVAL = 60秒
02                    // SCREEN_TYPE = 0x02 (5.89寸)
00                    // CUR_IMG_INDEX = 0
[CRC16]
```

6.7.4 获取播放配置 (CMD = 0x02)

请求 PAYLOAD: 空

响应 PAYLOAD:

字段	字节数	说明
PLAY_MODE	1	播放模式: 0x00 顺序, 0x01 随机, 0x02 手动
INTERVAL	4	切换间隔, 单位秒

示例:

请求: AA 02 00 00 [CRC16]

响应: AA 7F 07 00

```
02                    // ACK_CMD (对应0x02)
00                    // RESULT = 0x00 (成功)
01                    // PLAY_MODE = 0x01 (随机播放)
0F 00 00 00          // INTERVAL = 15秒
[CRC16]
```

6.7.5 获取软件版本 (CMD = 0x03)

请求 PAYLOAD: 空

响应 PAYLOAD:

字段	字节数	说明
SW_VER	64	软件版本字符串

示例:

请求: AA 03 00 00 [CRC16]

响应: AA 7F 42 00

```
03          // ACK_CMD (对应0x03)
00          // RESULT = 0x00 (成功)
46 57 5F 56 31 2E 30 2E // "FW_V1.0."
35 5F 32 30 32 36 30 34 // "5_202604"
30 32 00 00 ... [补零] // "02\0\0..." (共64字节)
[CRC16]
```

6.7.6 获取电量信息 (CMD = 0x04)

请求 PAYLOAD: 空

响应 PAYLOAD:

字段	字节数	说明
BATTERY	1	电量百分比, 范围 0~100

当前固件不支持充电状态检测, 因此该命令只返回电量百分比。

示例:

请求: AA 04 00 00 [CRC16]

响应: AA 7F 03 00

```
04          // ACK_CMD (对应0x04)
00          // RESULT = 0x00 (成功)
50          // BATTERY = 80%
[CRC16]
```

6.7.7 获取连接间隔 (CMD = 0x05)

请求 PAYLOAD: 空

响应 PAYLOAD:

字段	字节数	说明
CONN_INTERVAL	2	BLE 连接间隔, 单位 1.25ms, 范围 6~3200

说明:

- CONN_INTERVAL 使用小端格式。
- 实际时间 = CONN_INTERVAL * 1.25ms, 即范围为 7.5ms ~ 4000ms。

示例:

请求: AA 05 00 00 [CRC16]

响应: AA 7F 04 00

05 // ACK_CMD (对应0x05)
00 // RESULT = 0x00 (成功)
F0 00 // CONN_INTERVAL = 240, 即300ms
[CRC16]

6.7.8 设置播放配置 (CMD = 0x10)

请求 PAYLOAD:

字段	字节数	说明
PLAY_MODE	1	0x00 顺序, 0x01 随机, 0x02 手动
INTERVAL	4	切换间隔, 单位秒, 范围 60 ~ 4294967295

响应 PAYLOAD:

字段	字节数	说明
RESULT	1	结果码 (见6.6.1)
IMG_MASK	12	当前图片掩码

说明: 成功后设备更新配置区, 返回当前图片掩码用于 APP 状态确认。

示例:

请求: AA 10 05 00

02 // PLAY_MODE = 0x02 (手动播放)
3C 00 00 00 // INTERVAL = 60秒
[CRC16]

响应: AA 7F 0E 00

10 // ACK_CMD (对应0x10)
00 // RESULT = 0x00 (成功)
07 00 00 00 00 00 00 00 00 00 00 00 // IMG_MASK
[CRC16]

6.7.9 设置系统时间 (CMD = 0x11)

请求 PAYLOAD:

字段	字节数	说明
YEAR	2	年份 (小端), 范围 2000 ~ 2100
MONTH	1	月份, 范围 1 ~ 12
DAY	1	日期, 范围 1 ~ 31
HOURL	1	小时, 范围 0 ~ 23

字段	字节数	说明
MINUTE	1	分钟，范围 0 ~ 59
SECOND	1	秒，范围 0 ~ 59

响应 PAYLOAD： 返回通用 ACK（CMD=0x7F，包含 RESULT 状态码）

说明： 用于 APP 同步设备系统时间，设备收到后更新 RTC 时钟。

示例：

请求：AA 11 08 00	
E8 07	// YEAR = 2024（小端）
04	// MONTH = 4
15	// DAY = 21
0A	// HOUR = 10
1E	// MINUTE = 30
00	// SECOND = 0
[CRC16]	
响应：AA 7F 02 00	
11	// ACK_CMD（对应0x11）
00	// RESULT = 0x00（成功）
[CRC16]	

6.7.10 删除图片（CMD = 0x12）

请求 PAYLOAD：

字段	字节数	说明
IMG_INDEX_MASK	12	要删除的图片索引掩码（位运算：欲删除图片0和2，则传 0x00000005）

响应 PAYLOAD：

字段	字节数	说明
RESULT	1	结果码（见6.6.1）
IMG_MASK	12	更新后的图片掩码

错误处理：

- IMG_INDEX_MASK 长度必须为 12 字节，4 字节旧格式不再支持；若长度不是 12 字节，返回 0x02。
- 删除掩码为空，或包含当前屏幕容量以外的 bit，返回 0x01。
- 删除掩码包含设备当前不存在的图片 bit，返回 0x07。

示例：

```

请求: AA 12 0C 00
      05 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 // IMG_INDEX_MASK
      [CRC16]

响应: AA 7F 0E 00
      12 // ACK_CMD (对应0x12)
      00 // RESULT = 0x00 (成功)
      02 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 // IMG_MASK
      [CRC16]

```

6.7.11 设置连接间隔 (CMD = 0x13)

请求 PAYLOAD:

字段	字节数	说明
CONN_INTERVAL	2	BLE 连接间隔, 单位 1.25ms, 范围 6~3200

响应 PAYLOAD: 返回通用 ACK (CMD=0x7F, 包含 RESULT 状态码)

说明:

- CONN_INTERVAL 使用小端格式。
- 实际时间 = CONN_INTERVAL * 1.25ms, 即范围为 7.5ms ~ 4000ms。
- 若 CONN_INTERVAL 小于 6 或大于 3200, 设备返回 0x01 参数错误。
- 设置成功后, 设备保存连接间隔配置, 并按该值更新当前或下一次 BLE 连接参数。

示例:

```

请求: AA 13 02 00
      F0 00 // CONN_INTERVAL = 240, 即300ms
      [CRC16]

响应: AA 7F 02 00
      13 // ACK_CMD (对应0x13)
      00 // RESULT = 0x00 (成功)
      [CRC16]

```

6.7.12 屏幕刷新 (CMD = 0x24)

请求 PAYLOAD:

字段	字节数	说明
IMG_INDEX	1	要显示的图片索引, 范围由屏幕容量决定; 0xFF 表示保持当前显示, 不触发刷新

响应 PAYLOAD:

字段	字节数	说明
RESULT	1	结果码（见6.6.1）；若图片不存在返回 0x05
CUR_IMG_INDEX	1	当前显示的图片索引

说明：

- 屏幕刷新是**显式指令**，设备应立即更新屏幕显示指定的图片
- 若指定图片不存在（IMG_MASK 对应位为0）或索引越界，当前固件返回错误代码 0x05
- 若 IMG_INDEX=0xFF，设备保持上次显示，返回当前有效图片索引；若当前没有有效图片，返回 0xFF
- 此命令可用于图片传输完成后刷新显示，也可在任何时候切换显示

示例：

```
请求：AA 24 01 00
      02                      // IMG_INDEX = 2
      [CRC16]

响应（成功）：AA 7F 03 00
      24                      // ACK_CMD（对应0x24）
      00                      // RESULT = 0x00（成功）
      02                      // CUR_IMG_INDEX = 2
      [CRC16]

响应（图片不存在）：AA 7F 03 00
      24                      // ACK_CMD（对应0x24）
      05                      // RESULT = 0x05（图片不存在）
      FF                      // CUR_IMG_INDEX = 0xFF（未更改）
      [CRC16]
```

6.8 图片传输协议

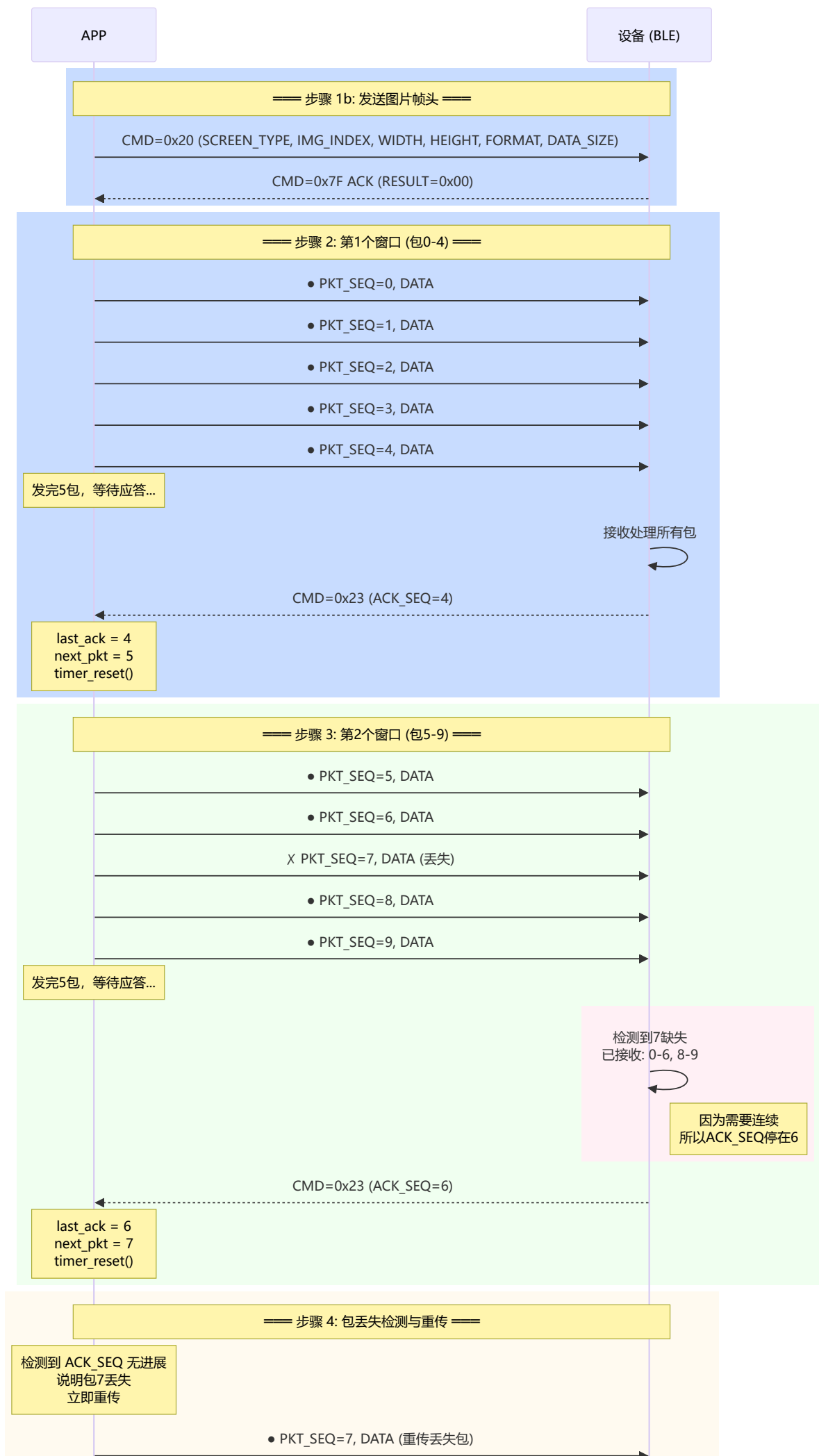
每张图片的索引唯一，从 0 开始，依次递增。

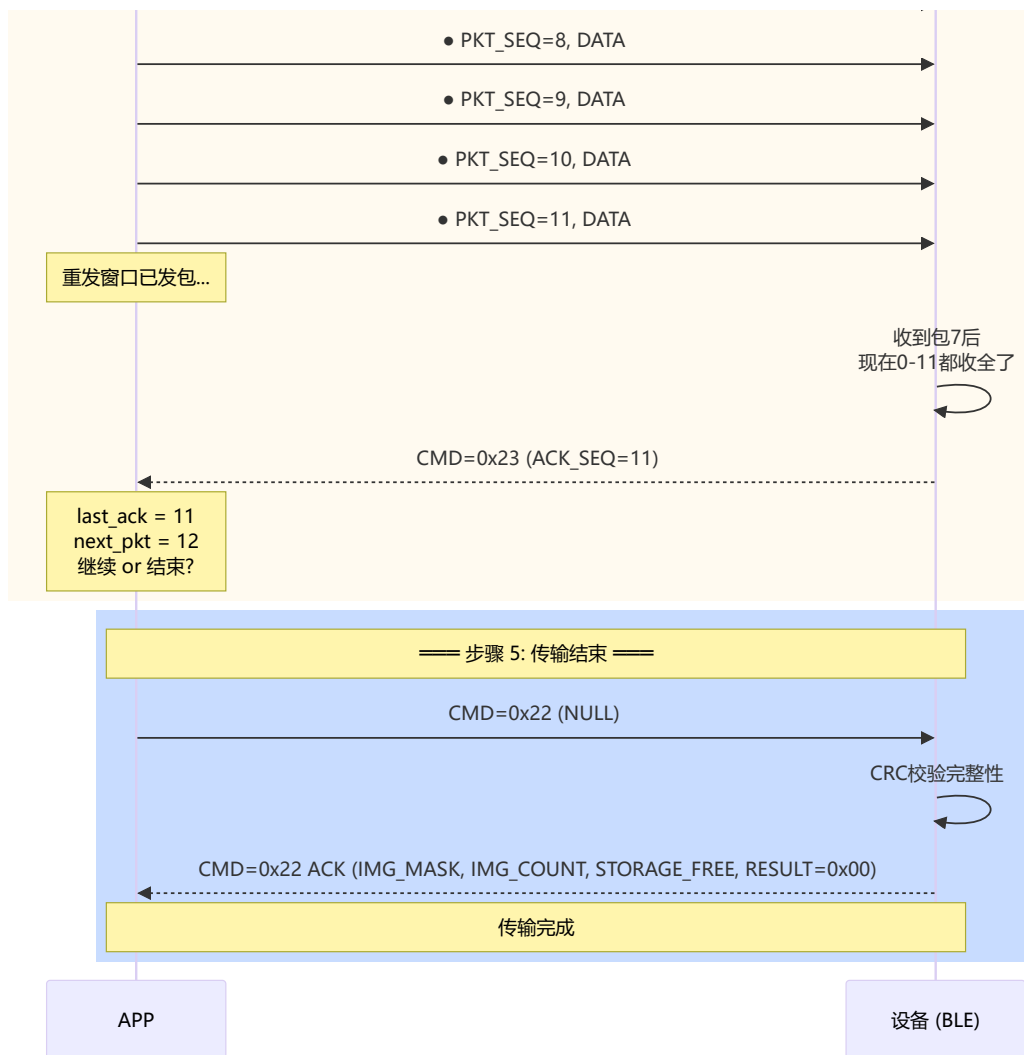
传输机制统一为"图片帧头 + 窗口分包传输 + 累计确认 + 文件结束"。

传输流程概述：

1. **图片帧头**（CMD=0x20）：发送屏幕类型、图片索引、宽度、高度、格式、数据大小、CRC32
2. **数据分包传输**（CMD=0x21）：使用窗口机制（5包）分片传输图片数据
3. **文件结束**（CMD=0x22）：发送结束信号，设备完成写入并更新 IMG_MASK
4. **屏幕刷新**（可选CMD=0x24）：指定要显示的图片索引

窗口大小为 5 包，APP 每发送 5 个包后必须等待设备应答。设备仅返回已连续接收的最后包号，APP 从该号的下一个包开始发送。若检测到 ACK_SEQ 无进展（包丢失信号），APP 重新发送未确认的包。若 1s 未收到应答包，表示连接失败，中断图片传输。





流程说明:

- **窗口机制**: 每次发5个包, 必须等待应答
- **累计确认**: ACK_SEQ 表示已按序接收到的最后包号
- **简洁高效**: 无需显式告知哪个包丢失, APP 自动推导

6.8.1 开始传输 (CMD = 0x20)

图片帧头

请求 PAYLOAD:

字段	字节数	说明
SCREEN_TYPE	1	屏幕类型: 0x01 3.7寸, 0x02 5.89寸; 必须与设备当前固件编译类型一致
IMG_INDEX	1	图片索引, 范围由屏幕容量决定
WIDTH	2	图片宽度 (像素)
HEIGHT	2	图片高度 (像素)
FORMAT	1	当前固件仅支持 0x01 六色 4bpp 原始帧缓存

字段	字节数	说明
IMG_DATA_SIZE	4	该图片的数据长度
IMG_CRC32	4	图片数据的 CRC32-MPEG2

图片数据格式 (FORMAT = 0x01)

图片数据为六色 4bpp 原始数据。每个像素占 4 bit，每个字节表示左右相邻的 2 个像素：

Byte bit	内容
bit7~bit4	左侧像素颜色值
bit3~bit0	右侧像素颜色值

颜色值：

颜色	值
黑	0x0
白	0x1
黄	0x2
红	0x3
蓝	0x5
绿	0x6

5.89 寸图片数据

帧头参数：

字段	值
SCREEN_TYPE	0x02
WIDTH	680
HEIGHT	960
IMG_DATA_SIZE	326400

数据排列：

项	规则
行数	960 行
每行像素	680 像素

项	规则
每行字节	340 字节
行顺序	从屏幕顶部到屏幕底部
行内顺序	从屏幕左侧到屏幕右侧
每字节像素	高 4 bit 为左像素，低 4 bit 为右像素

也就是：

第 0~339 字节	= 第 0 行
第 340~679 字节	= 第 1 行
...	
第 326060~326399 字节	= 第 959 行

3.7 寸图片数据

帧头参数：

字段	值
SCREEN_TYPE	0x01
WIDTH	480
HEIGHT	720
IMG_DATA_SIZE	172800

注意：3.7 寸帧头宽高固定为 480x720，不要写成 720x480。

3.7 寸图片数据按横向源图存放：

项	规则
数据行数	480 行
每行像素	720 像素
每行字节	360 字节
行顺序	从横向源图顶部到横向源图底部
行内顺序	从横向源图左侧到横向源图右侧
每字节像素	高 4 bit 为左像素，低 4 bit 为右像素

横向源图与最终屏幕显示方向的关系：

横向源图	最终 3.7 寸屏幕显示
整张源图	顺时针旋转 90° 后显示到屏幕
源图左上角	屏幕左下角

横向源图	最终 3.7 寸屏幕显示
源图右上角	屏幕左上角
源图左下角	屏幕右下角
源图右下角	屏幕右上角

也就是：

第 0~359 字节	= 横向源图第 0 行
第 360~719 字节	= 横向源图第 1 行
...	
第 172440~172799 字节	= 横向源图第 479 行

示例：若最终屏幕要显示一张正常竖向 480x720 图片，发送前应先将该图片逆时针旋转 90°，得到 720x480 横向源图，再按上表打包。

响应 PAYLOAD： 返回通用 ACK (CMD=0x7F, 包含 RESULT 状态码)

发送规则：每次传输前，APP 发送一次图片帧头，指定屏幕类型和要传输的图片索引，等待设备 ACK 后再开始数据传输。

错误处理：

- SCREEN_TYPE、WIDTH、HEIGHT、FORMAT、IMG_DATA_SIZE 或 IMG_INDEX 不符合当前固件编译屏幕参数时，返回 0x01。
- 指定 IMG_INDEX 已存在图片时，返回 0x07，APP 应先通过 0x12 删除旧图再重新上传。
- 固件传输状态未空闲时，返回 0x0B。
- Flash/缓存初始化失败时，返回 0x04。

示例：

请求：AA 20 0F 00	
02	// SCREEN_TYPE = 0x02 (5.89寸)
00	// IMG_INDEX = 0
A8 02	// WIDTH = 680
C0 03	// HEIGHT = 960
01	// FORMAT = 0x01 (六色原始)
00 FA 04 00	// IMG_DATA_SIZE = 326400字节
12 34 56 78	// IMG_CRC32
[CRC16]	
响应：AA 7F 02 00	
20	// ACK_CMD (对应0x20)
00	// RESULT = 0x00 (成功)
[CRC16]	
响应（设备忙）：AA 7F 02 00	
20	// ACK_CMD (对应0x20)
0B	// RESULT = 0x0B (设备忙)
[CRC16]	

6.8.2 传输数据包 (CMD = 0x21)

请求 PAYLOAD:

字段	字节数	说明
PKT_SEQ	2	包序号, 从 0 开始递增
DATA	N	图片数据 (最多 236 字节; 因此 0x21 的 PAYLOAD 最大为 238 字节, 包含 2 字节 PKT_SEQ)

DATA 是 IMG_DATA 的连续切片:

```
start = PKT_SEQ * 236
len = min(236, IMG_DATA_SIZE - start)
DATA = IMG_DATA[start : start + len]
```

除最后一包外, 每包 DATA 长度必须为 236 字节。最后一包长度按剩余数据计算。

响应 PAYLOAD: 返回图片传输应答 (CMD=0x23, 包含 ACK_SEQ)

发送规则 (窗口机制):

- 1. APP 一次连续发送最多 5 个数据包, 之后**必须等待设备应答**
- 2. 设备返回 ACK_SEQ (已连续接收到的最后包号)
- 3. APP 根据 ACK_SEQ 计算下一个要发的包号: `next_seq = ACK_SEQ + 1`
- 4. 若 ACK_SEQ 无进展 (一直是同一数值), 说明有丢包, 超时后重新发送该包及之后的包

示例:

```
请求1: AA 21 EE 00
      00 00                // PKT_SEQ = 0
      [236字节的图片数据]
      [CRC16]

请求2: AA 21 EE 00
      01 00                // PKT_SEQ = 1
      [236字节的图片数据]
      [CRC16]

...发送5个包后等待应答...

响应: AA 23 02 00
      04 00                // ACK_SEQ = 4 (已连续接收包0~4)
      [CRC16]

若包7丢失, 收到的应答为:
      06 00                // ACK_SEQ = 6 (已连续接收包0~6)
```

6.8.3 传输应答 (CMD = 0x23)

完整说明详见“6.6.2 图片传输应答 (CMD = 0x23) ”。设备在收到每个 5 包窗口的数据后返回此应答，告知 APP 已连续接收到的最后包号。

6.8.4 结束传输 (CMD = 0x22)

请求 PAYLOAD:

NULL (无数据)

响应 PAYLOAD:

字段	字节数	说明
RESULT	1	传输结果 (0x00=成功, 其他见6.6.1)
IMG_MASK	12	更新后的图片掩码 (所有新增图片位已置1)
IMG_COUNT	2	当前总图片数量
STORAGE_FREE	4	剩余可用字节数

设备处理过程:

- 1. 校验所有接收数据长度是否 = IMG_DATA_SIZE
- 2. 校验整个图片完整性 (CRC32)
- 3. 将图片按指定 IMG_INDEX 写入 Flash, 更新对应的 IMG_MASK 位
- 4. 更新存储信息, 返回最终结果

示例:

```
请求: AA 22 00 00 [CRC16]

响应 (成功): AA 7F 14 00
    22                // ACK_CMD (对应0x22)
    00                // RESULT = 0x00 (成功)
    01 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 // IMG_MASK
    01 00                // IMG_COUNT = 1
    00 00 C0 00          // STORAGE_FREE = 12582912字节
    [CRC16]

响应 (CRC校验失败): AA 7F 02 00
    22                // ACK_CMD (对应0x22)
    0A                // RESULT = 0x0A (校验失败)
    [CRC16]
```

6.8.5 包丢失与重传机制

工作流程:

- 1. APP 发送 5 个包 (窗口) , 等待应答
- 2. 设备返回 ACK_SEQ (已连续接收到的最后包号)
- 3. 调整窗口下界: 下次从 ACK_SEQ + 1 开始发送

4. **包丢失重传**：若 ACK_SEQ 无进展（保持不变），说明有丢包，APP 立即重新发送窗口内的包；若 1s 未收到应答包，表示连接失败，中断图片传输

状态跟踪：

```
last_ack_seq = -1  // 上次确认的包号

发送包0-4 → 收到 ACK_SEQ=4
last_ack_seq = 4, 下次发包5

发送包5-9 → 收到 ACK_SEQ=6（包7丢失）
last_ack_seq = 6, 下次发包7
由于 ACK_SEQ 没有推进到9，说明后续包有丢包

检测到丢包 → 立即重发包7-11
last_ack_seq = 6（没变），检测到 ACK_SEQ 无进展，立即触发重传

收到 ACK_SEQ=11
last_ack_seq = 11, 下次发包12
```

APP 实现建议：

- 记录 last_ack_seq，每次收到应答就检查 ACK_SEQ 是否有进展
- 若 ACK_SEQ 无进展（与上次相同），立即重传从 last_ack_seq+1 开始的 5 个包
- 保留所有已发送包的数据用于重传
- 若 1s 未收到任何应答包，说明连接失败，中断图片传输

6.8.6 屏幕刷新 (CMD = 0x24)

屏幕刷新命令详见 [6.7.12 屏幕刷新](#)。

在图片传输完成后，可使用此命令指定要显示的图片索引。该命令为可选项，APP 也可不发送此命令，由设备维持最后一次显示的图片。

6.8.7 替换图片

替换图片指删除旧的并上传新的。

APP指定要替换的图片索引，设备删除该图片并接收新的图片。

单张图片替换：



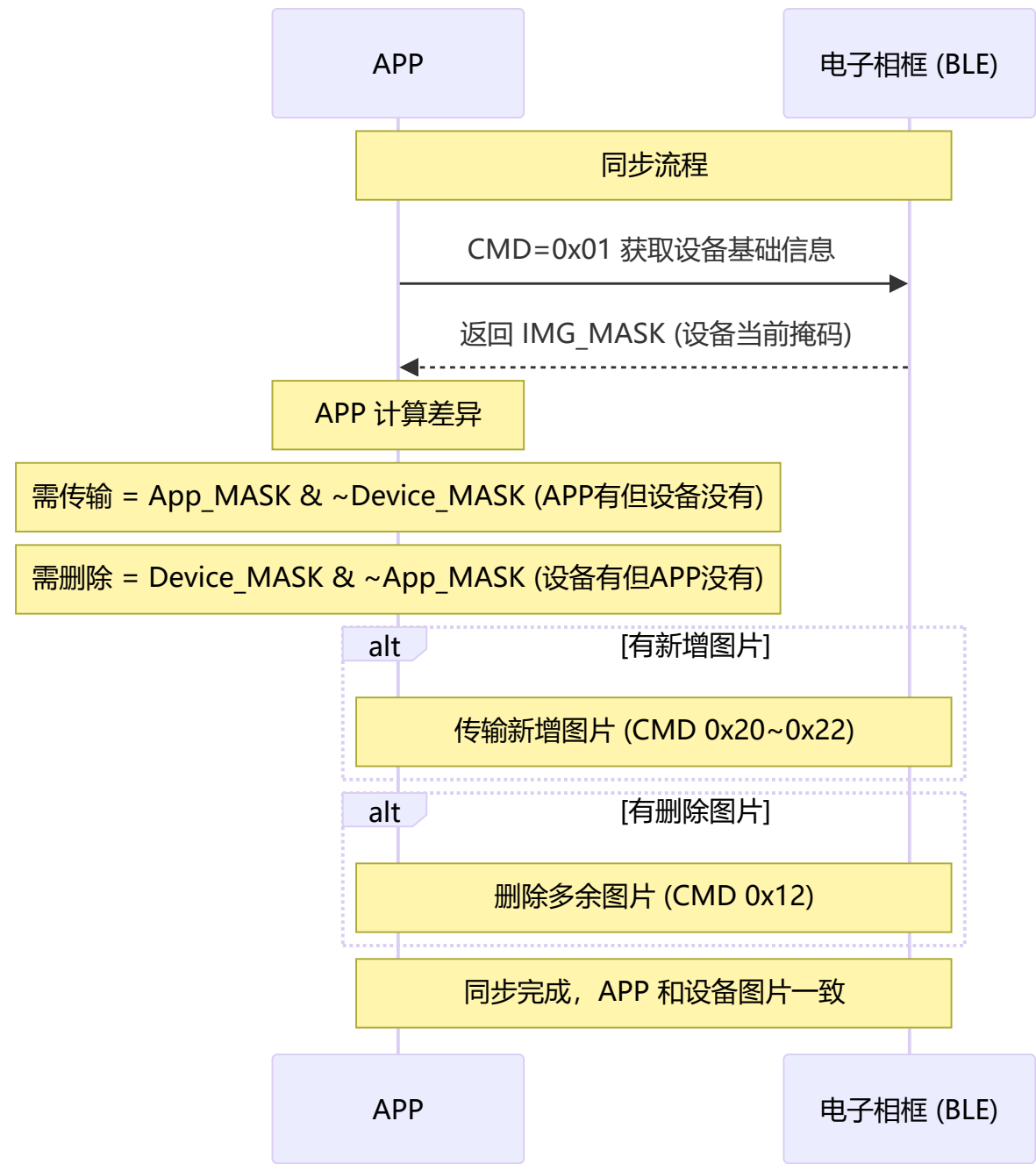
6.9 图片掩码同步机制

为了保证 APP 与设备之间的图片同步，采用 **12 字节图片掩码**。

6.9.1 图片掩码 (IMG_MASK) 规则

属性	说明
类型	12 字节数组，低字节在前
含义	每位对应一张图片 (bit0~bit95) ，1=存在，0=不存在
设备端	设备维护当前图片掩码，传输/删除图片时更新对应位

6.9.2 同步流程



同步原则:

- 1. APP 获取设备 IMG_MASK
- 2. APP 计算差异: 需传输 = App_MASK & ~Device_MASK, 需删除 = Device_MASK & ~App_MASK
- 3. APP 执行相应的传输/删除操作
- 4. 再次获取 IMG_MASK 确认同步完成

说明: IMG_MASK 是同步的核心, APP 应根据 IMG_MASK 的变化来管理设备中的图片。响应包中的 IMG_COUNT 为便利计算而保留 (可由 IMG_MASK 置位数计算得出), 但减少了 APP 端的运算开销。

6.10 广播规范

6.10.1 广播目标

- 1. 被 APP / 小程序快速发现
- 2. 区分设备类型和屏幕尺寸
- 3. 表示设备当前处于可连接、未被占用状态

6.10.2 广播类型

类型	说明
普通工作状态	可连接广播 <code>ADV_IND</code>

6.10.3 广播生命周期

阶段	广播行为	APP / 小程序侧说明
未连接	设备开启唯一的可连接广播	APP 可扫描并发起连接
已连接	设备自动关闭广播	设备已被当前手机端占用，其他手机端无法扫描到或连接该设备
连接断开	设备重新开启可连接广播	其他手机端可重新扫描并连接

连接占用说明： 设备同一时间只维护一个 BLE 连接。连接期间不再开启广播，也不支持第二个 Central 同时连接。手机端若需要切换控制设备，应先主动断开当前连接，等待设备重新广播后再由新的手机端连接。

6.10.4 广播间隔建议

1 秒广播

6.10.5 广播名称规范

屏幕类型	广播名称
3.7 寸	<code>EF6-370</code>
5.89 寸	<code>EF6-589</code>

说明： 设备唯一性通过 Manufacturer Data 的 Device_ID 或 MAC 地址区分。

6.10.6 广播数据格式

广播包结构如下：

- Flags
- Complete Local Name
- Manufacturer Specific Data

6.10.7 Manufacturer Data 内容定义

字段	字节数	说明
Company_ID	2	公司标识， 0xFFFF
Screen_Type	1	屏幕类型： 0x01 3.7寸， 0x02 5.89寸， 0x03 7.3寸
Device_ID	4	设备标识符
Battery	1	电池电量百分比（0-100）

文档版本

版本	日期	说明
1.0	2026-04-03	初稿发布版：完整的6色电子相框PRD 包含累积ACK协议、图片掩码同步机制、单张图片传输协议
1.1	2026-04-22	新增 CMD=0x11 设置系统时间指令
1.2	2026-05-20	更新 3.7/5.89 分辨率、12 字节 IMG_MASK、0x20 SCREEN_TYPE 帧头、236 字节分包和相关应答长度；移除充电状态检测；明确 Busy、删除、刷新、超时和错误码规则；新增设备单连接说明：连接后关闭广播，断开后重新广播，同一时间仅允许一个手机端连接
1.3	2026-05-21	明确 FORMAT=0x01 六色 4bpp 图片数据格式；补充 3.7 寸 landscape_cw 源图流说明：协议宽高仍为 480x720，图片数据按 720x480 行优先存储
1.4	2026-06-09	新增 CMD=0x05 获取连接间隔、CMD=0x13 设置连接间隔；连接间隔单位为 1.25ms，范围 6 ~ 3200
1.5	2026-07-03	新增 OTA 第一包 128 字节头信息握手规范